

TABLE OF CONTENTS

CHAPTER NO	TITLE	PAGE NO
	ABSTRACT	
	LIST OF FIGURE	
	LIST OF TABLE	
	LIST OF ABBREVIATION	
1	INTRODUCTION	1
	1.1 Background	1
	1.2 Problem Statement	2
	1.3 Objectives of the Project	2
	1.4 Scope of the Project	3
	1.5 Organisation of the Report	3
2	LITERATURE REVIEW	4
	2.1 WebScrapingTechnologies	4
	2.2 AI-PoweredDataProcessing	4
	2.3 LargeLanguageModelsforTextProcessing	5
	2.4 Multi-FormatDocumentGeneration	5
	2.5 WebApplicationFrameworks	6
	2.6 GapsinExistingSystems	6
3	SYSTEM REQUIREMENTS AND ANALYSIS	7
	3.1 Functional Requirements	7
	3.2 Non-Functional Requirements	7
	3.3 Hardware Requirements	8
	3.4 Software Requirements	9
4	SYSTEM DESIGN AND ARCHITECTURE	11
	4.1 SystemArchitectureOverview	11
	4.2 ModuleDescriptions	11
	4.3 DataFlowDesign	13
	4.4 OverallSystemFlowchart	13
	4.5 DatabaseandStorageDesign	13
5	IMPLEMENTATION	14
	5.1 Development Environment Setup	14
	5.2 Flask Application (app.py)	14
	5.3 Scraper Module (scraper.py)	15
	5.4 LangChain Helper (langchain_helper.py)	16

	5.5 Export Utilities (export_utils.py)	16
	5.6 Frontend Interface	17
	5.7 Technology Stack Summary	18
6	TESTING AND EVALUATION	19
	6.1 Testing Strategy	19
	6.2 Unit Testing	18
	6.3 Integration Testing	22
	6.4 Functional Test Cases	22
	6.5 Performance Evaluation	23
7	RESULTS AND DISCUSSION	25
	7.1 Scraping Results	25
	7.2 AI Processing Results	25
	7.3 Export Format Results	26
	7.4 Comparison with Existing Systems	26
	7.5 Limitations	27
8	APPENDIX	28
	8.1 Source Code	28
	8.2 Screenshots	33
9	CONCLUSION AND FUTURE WORK	36
	9.1 Conclusion	36
	9.2 Future Enhancements	36
	9.3 Summary	37
	REFERENCES	37

LIST OF TABLES

TABLE NO	TABLES NAME	PAGE NO
3.3	Hardware Requirements	8
3.4	Software Requirement	9
4.2	Module Description	11
5.7	Technology Stack Summary	18
6.2.1	Flask Rotate Testing	19
6.2.2	Scraper Module Testing	20
6.2.3	Langchain Helper Testing	21
6.2.4	Export Module Testing	21
6.4	Functional Test Cases	22
6.5	Performance Evaluation	23
7.4	Comparison with Existing System	26

LIST OF FIGURES

FIGURE	FIGURE NAME	PAGE NO
8.2.1	AI Web Scraper Interface	33
8.2.2	Web Scrapping Interface with sample URL	33
8.2.3	Content Extraction stage	34
8.2.4	Data Display stage	34
8.2.5	Generated output from Scrapped data	35
8.2.6	AI-powered web scraping interface with Data processing and export options	35

LIST OF ABBREVIATIONS

SNO	ACRONYMS	FULL FORM
1		
2	AI	Artificial Intellingence
3	LLM	Large Language Model
4	NLP	Natural Language Processing
5	API	Application Programming Interface
6	HTML	HyperText Markup Language
7	CSS	Cascading Style Sheets
8	DOM	Document Object Model
9	JSON	JavaScript Object Notation
10	CSV	Comma-Seperated Values
11	REST	Representational State Transfer
12	HTTP	HyperText Transfer Protocal
13	URL	Uniform Resource Locator
14	XPath	XML Path Language
15	JS	JavaScript
16	GPT	Generative Pre-trained Interface
17	CLI	Command Line Interface
18	ORM	Object Relational Mapping
	UI	User Interface

ABSTRACT

The rapid proliferation of web-based data has created an urgent demand for intelligent systems capable of extracting, processing, and structuring information from online sources automatically. Traditional web scraping tools provide raw data extraction but lack the cognitive capability to interpret, filter, and summarise content according to user intent. This project presents Scraper AI, a full-stack AI-powered web scraping and content intelligence platform that combines automated browser-based data extraction with Large Language Model (LLM)-driven content processing.

Scraper AI is developed as a web application using the Flask framework and Python, providing an intuitive browser-based interface through which users can submit any publicly accessible URL for automated content extraction. The scraping engine is built on Selenium WebDriver operating in headless Chrome mode, enabling dynamic JavaScript-rendered page handling that static HTTP request-based scrapers cannot achieve.

The intelligent content processing pipeline is powered by Claude 3 Sonnet (Anthropic) accessed via the OpenRouter API and integrated through the LangChain framework. Users provide natural language instructions alongside the scraped content, and the LLM interprets, filters, summarises, or reformats the data according to the specified prompt. This transforms raw webpage text into structured, actionable information without manual data processing.

A significant feature of the system is its multi-format export capability, implemented through the `export_utils.py` module. Processed content can be exported as Word documents (.docx), Excel spreadsheets (.xlsx), plain text (.txt), PDF files (.pdf), PowerPoint presentations (.pptx), and XML documents (.xml), making it versatile for diverse professional and academic workflows.

The frontend interface, built with HTML5, CSS3, and vanilla JavaScript, provides a clean, responsive user experience. The application is deployed on Vercel, making it accessible as a live web service at <https://scraper-ai-ten.vercel.app>.

This project demonstrates the practical convergence of web automation, AI-driven natural language processing, and multi-format data management in a unified, production-ready application.

Keywords: *Web Scraping, Selenium, Flask, LangChain, OpenRouter, LLM, Content Extraction, NLP, Export Utilities, Python, Vercel Deployment*

CHAPTER 1

INTRODUCTION

1.1 Background

The World Wide Web has evolved into the most comprehensive repository of human knowledge, commerce, news, research, and public discourse ever assembled. As of 2025, the internet hosts over 1.9 billion active websites, generating petabytes of new data every day across news portals, e-commerce platforms, research databases, government portals, and social media. This volume of publicly accessible information presents an extraordinary opportunity for automated data collection and intelligent analysis across virtually every professional domain.

Web scraping — the automated extraction of content from web pages — has emerged as a critical technique for harvesting this data at scale. However, traditional web scraping tools such as BeautifulSoup, Scrapy, and simple HTTP request-based parsers are increasingly inadequate in a modern web landscape. Contemporary websites rely heavily on JavaScript frameworks such as React, Vue, and Angular, which render content dynamically in the browser after the initial HTML document is loaded. Static HTTP parsers cannot access this rendered content, rendering them ineffective for a significant proportion of modern web targets.

The emergence of Large Language Models (LLMs) such as GPT-4, Claude, and LLaMA has introduced a transformative capability: natural language understanding and generation at a level that enables intelligent interpretation of scraped content. Rather than simply extracting raw text, LLM-integrated systems can interpret user intent expressed in plain English, filter relevant information from noisy web content, summarise long-form articles, restructure tabular data, and generate structured documents — all from a conversational prompt.

This confluence of browser automation, AI-powered NLP, and flexible export capabilities forms the technical foundation of Scraper AI. The project addresses the practical needs of researchers, data analysts, content writers, business intelligence professionals, and students who require intelligent web data extraction without deep technical expertise in web scraping or data processing.

1.2 Problem Statement

Existing web scraping tools and data extraction platforms face several fundamental limitations that restrict their utility for non-technical users and complex data workflows. First, most scraping tools operate at the HTML level and cannot handle JavaScript-rendered content, making them incompatible with a growing majority of modern web applications. Second, even when raw data is successfully extracted, it is typically unstructured, noisy, and requires significant post-processing effort to become useful for analysis or reporting.

Third, there is a pronounced gap between data extraction tools and intelligent content understanding. Existing platforms require users to write custom parsing logic, regular expressions, or CSS selectors to target specific page elements — a technically demanding approach that is inaccessible to many potential users. Fourth, the output formats of most scrapers are limited to CSV or JSON, which may not align with the downstream workflows of users who need Word documents, presentations, or spreadsheets.

There is therefore a significant need for an integrated platform that combines dynamic JavaScript-capable web scraping, conversational AI-driven content processing, and flexible multi-format document export in a user-friendly web application interface.

1.3 Objectives of the Project

- To design and develop an AI-powered web scraping platform that can extract content from any publicly accessible URL including JavaScript-rendered pages using Selenium WebDriver.
- To integrate an LLM-based content processing pipeline using LangChain and Claude 3 Sonnet (via OpenRouter) that can interpret natural language user prompts and intelligently filter, summarise, or restructure scraped content.
- To implement a multi-format export system that converts processed content into Word (.docx), Excel (.xlsx), PDF (.pdf), PowerPoint (.pptx), plain text (.txt), and XML (.xml) formats.

- To develop an intuitive, responsive web interface using HTML5, CSS3, and JavaScript that enables non-technical users to operate the full scraping and export pipeline without coding.
- To deploy the application as a publicly accessible web service using Vercel for cloud hosting.

1.4 Scope of the Project

The scope of Scraper AI encompasses the full pipeline from URL submission to intelligent document generation. The system targets publicly accessible web pages that do not require authentication. It is designed to handle both static HTML content and JavaScript-rendered dynamic pages through headless Chrome automation.

The AI processing layer supports a broad range of natural language tasks on scraped content: summarisation, key information extraction, data restructuring, sentiment extraction, entity recognition, and content classification. The export functionality covers six output formats suitable for professional, academic, and business workflows.

The system is designed as a multi-user web application accessible via any modern web browser. It does not persist scraped data beyond the active session, ensuring privacy compliance. The deployment on Vercel enables global accessibility with low-latency response for the Flask API endpoints.

1.5 Organisation of the Report

This report is structured across nine chapters. Chapter 2 provides a comprehensive review of existing literature on web scraping, LLM-based NLP, and web application development. Chapter 3 details the system requirements, both functional and non-functional. Chapter 4 presents the system architecture and module design. Chapter 5 elaborates on the implementation details of each module. Chapter 6 covers the testing strategy and evaluation results. Chapter 7 discusses performance results and comparisons with existing systems. Chapter 8 presents source code excerpts and screenshots. Chapter 9 concludes the project with future work directions.

CHAPTER 2

LITERATURE REVIEW

2.1 Web Scraping Technologies

Web scraping has a research history spanning over two decades. Early scraping techniques relied on simple HTTP GET requests combined with HTML parsers such as BeautifulSoup (Richardson, 2007) to extract content from static HTML documents. These approaches were effective for the web of the early 2000s but have become progressively less applicable as JavaScript-driven Single Page Applications (SPAs) have come to dominate the modern web landscape.

Selenium WebDriver, originally developed by Jason Huggins at ThoughtWorks in 2004 as a test automation tool, has since been widely adopted as a web scraping framework for JavaScript-rendered content. Mitchell (2018) demonstrated that Selenium-based scrapers can interact with dynamic web applications at a level that approximates real user behaviour, including clicking, scrolling, and form submission. The introduction of headless browser modes — particularly headless Chrome via the ChromeDriver interface — removed the need for a display server, enabling Selenium to run efficiently in server environments and CI/CD pipelines.

Playwright (Denikhin et al., 2020) and Puppeteer (Google Chrome Team, 2017) represent more recent alternatives to Selenium for browser automation. These tools offer improved performance and more modern APIs, but Selenium remains the most widely documented and community-supported framework, particularly in Python ecosystems. This project employs Selenium with ChromeDriver in headless mode as the scraping engine.

2.2 AI-Powered Data Processing

The integration of machine learning and artificial intelligence into data processing pipelines has been an active research area since at least the early 2010s. Early work focused on applying statistical NLP techniques — TF-IDF weighting, named entity recognition (NER), and sentiment analysis — to post-process scraped web content. Feldman and Sanger (2007) provided foundational approaches for automated text mining from web sources.

The advent of pre-trained transformer models beginning with BERT (Devlin et al., 2018) and accelerating through the GPT series (Radford et al., 2018; Brown et al., 2020) dramatically expanded the capability of automated text processing systems. These models, pre-trained on massive text corpora, can be prompted to perform a wide variety of text understanding and generation tasks without task-specific fine-tuning. This zero-shot and few-shot prompting capability is central to the Scraper AI content processing pipeline.

2.3 Large Language Models for Text Processing

The GPT-3 paper (Brown et al., 2020) established that large transformer models with sufficient parameters could perform complex NLP tasks such as summarisation, translation, question answering, and content generation through natural language prompting alone. Subsequent models including GPT-4 (OpenAI, 2023) and Anthropic's Claude series have continued to push the frontier of LLM capability, with Claude 3 Sonnet demonstrating particularly strong performance on reasoning, summarisation, and instruction-following benchmarks.

LangChain (Chase, 2022) is an open-source framework that abstracts and orchestrates LLM interactions through standardised interfaces. It provides PromptTemplate, LLMChain, and agent abstractions that simplify the integration of LLMs into production applications. The framework supports multiple LLM providers through a unified API, enabling seamless switching between models. This project uses LangChain with the ChatOpenAI class configured to target the OpenRouter API, which provides access to Claude 3 Sonnet as the inference backend.

OpenRouter (openrouter.ai) functions as a unified LLM API gateway that aggregates access to multiple AI models from different providers including Anthropic, OpenAI, Google, and Meta under a single API endpoint compatible with the OpenAI SDK. This architecture simplifies model integration and provides fallback routing capabilities.

2.4 Multi-Format Document Generation

The automatic generation of structured documents in professional formats has been addressed through various Python libraries. python-docx (python-docx contributors, 2014) provides a comprehensive API for generating and editing Word documents programmatically, supporting paragraphs, tables, images, and styles. openpyxl (Gazoni & Clark, 2010) enables creation and

manipulation of Excel workbooks including formulae and charts. reportlab (Precedo, 2000) is a mature PDF generation library offering fine-grained control over page layout and typography.

For presentation generation, python-pptx (python-pptx contributors, 2013) provides a Python API for generating PowerPoint files with slides, text boxes, shapes, and embedded content. XML generation in Python is natively supported through the xml.etree module and the lxml library. The combination of these libraries in the export_utils.py module of Scraper AI provides a comprehensive multi-format document generation capability.

2.5 Web Application Frameworks

Flask (Ronacher, 2010) is a lightweight Python WSGI web application framework that follows the micro-framework philosophy — providing only the core components necessary for web development while remaining highly extensible. Flask's simplicity and flexibility make it particularly well-suited for building REST APIs and small-to-medium web applications. Its routing, request handling, and template rendering capabilities via Jinja2 cover the requirements of the Scraper AI backend.

For cloud deployment, Vercel (formerly ZEIT) provides a platform-as-a-service (PaaS) environment optimised for frontend frameworks and serverless functions. Vercel's automatic HTTPS provisioning, global CDN distribution, and Git-based continuous deployment pipeline make it an efficient choice for deploying Flask APIs and static frontend assets.

2.6 Gaps in Existing Systems

A review of existing web scraping and data extraction platforms — including Octoparse, ParseHub, Import.io, Scrapy Cloud, and Apify — reveals that while they provide sophisticated scraping capabilities, they uniformly lack integrated LLM-powered content intelligence. These platforms extract data as structured tables or CSV files but cannot interpret user intent expressed in natural language or generate formatted documents automatically.

Conversely, AI writing and summarisation tools such as Notion AI, Jasper, and Copy.ai can process text intelligently but do not provide web scraping capabilities. The user must manually copy content from web pages into these tools, a friction-inducing step that the Scraper AI pipeline eliminates entirely.

CHAPTER 3

SYSTEM REQUIREMENTS AND ANALYSIS

3.1 Functional Requirements

The functional requirements define the observable behaviours and capabilities of the Scraper AI system:

- FR-01: The system shall accept a URL input from the user through the web interface for automated content extraction.
- FR-02: The system shall scrape the full text content of the target URL using a headless Chrome browser driven by Selenium WebDriver.
- FR-03: The system shall handle dynamically rendered JavaScript content by waiting for page load completion before extracting the body text.
- FR-04: The system shall display the raw scraped text content in the user interface for review.
- FR-05: The system shall accept a natural language prompt from the user describing the desired processing or extraction task.
- FR-06: The system shall submit the user prompt and scraped content to the LangChain-Claude AI pipeline for intelligent content processing.
- FR-07: The system shall display the AI-processed output in the user interface.
- FR-08: The system shall provide export functionality in six formats: .docx, .xlsx, .pdf, .pptx, .txt, and .xml.
- FR-09: The system shall generate a downloadable file in the selected format and make it accessible to the user.
- FR-10: The system shall handle errors in scraping (inaccessible URLs, bot detection) and AI processing (API failures) gracefully with informative error messages.

3.2 Non-Functional Requirements

- NFR-01 Performance: Scraping operations should complete within 10 seconds for typical web pages. AI processing responses should return within 15 seconds.

- NFR-02 Usability: The interface shall be operable by non-technical users without programming knowledge. All primary actions shall be accessible within two clicks from the homepage.
- NFR-03 Reliability: The system shall maintain 99% uptime leveraging Vercel's global CDN infrastructure.
- NFR-04 Compatibility: The web interface shall function correctly on Chrome, Firefox, Safari, and Edge browsers on desktop and mobile form factors.
- NFR-05 Security: No scraped content or user prompts shall be persistently stored in any server-side database. All data is session-scoped and discarded after the request lifecycle.
- NFR-06 Extensibility: The modular architecture shall support the addition of new AI models, scraping strategies, or export formats with minimal changes to existing code.
- NFR-07 Maintainability: All modules shall follow PEP 8 coding standards with docstrings and inline comments for maintainability.

3.3 Hardware Requirements

The following hardware specifications are recommended for running the development environment and server infrastructure:

Component	Specification
Processor	Intel Core i5 (8th Gen) or AMD Ryzen 5 equivalent or higher
RAM	Minimum 8 GB (16 GB recommended for multi-session Selenium operation)
Storage	50 GB SSD (for OS, Chrome, dependencies, and export file staging)
GPU	Not required (CPU-only inference via cloud LLM API)

Network	Broadband internet (minimum 10 Mbps for Selenium and API calls)
Display	1280 x 720 minimum for development interface

3.4 Software Requirements

The software stack required for development, testing, and deployment is as follows:

SoftwareComponent	Version/Details
OperatingSystem	Ubuntu22.04LTS/Windows10/macOS 12+
Python	Python3.10orhigher
Flask	3.0.x(WSGIwebframework)
Selenium	4.x(WebDriverautomation)
ChromeDriver	MatchingChromeversion(v120+)
LangChain	0.2.x(LLMorchestrationframework)
LangChain-Community	0.2.x(ChatOpenAIwrapper)
OpenRouterAPI	RESTAPIendpoint(api.anthropic.comvia openrouter.ai)
python-docx	1.1.x(Worddocumentgeneration)
openpyxl	3.1.x(Excelgeneration)
reportlab	4.x(PDFgeneration)

python-pptx	0.6.x (PowerPoint generation)
Node.js / npm	For Vercel deployment toolchain
Vercel CLI	Latest (cloud deployment)
Git	2.x (version control)
Browser	Google Chrome v120+ (for headless Selenium)

CHAPTER 4

SYSTEM DESIGN AND ARCHITECTURE

4.1 System Architecture Overview

Scraper AI employs a three-tier layered architecture comprising a client-side presentation tier, a server-side application tier, and external service integrations. The presentation tier is a browser-based frontend built with HTML5, CSS3, and vanilla JavaScript served by Flask's template rendering engine. The application tier consists of a Flask REST API server hosting four primary route handlers. External service integrations include the Selenium/ChromeDriver automation layer for web scraping and the OpenRouter/Claude API for AI inference.

The architecture follows a request-response pattern for all user interactions. Users submit requests through the frontend, which communicates with Flask endpoints via AJAX calls using the Fetch API. Flask routes orchestrate the appropriate scraping or processing pipeline and return JSON responses to the frontend for display and download.

4.2 Module Descriptions

The system is composed of five primary Python modules, each responsible for a distinct functional domain:

Module	File	Responsibility
Flask App	app.py	Route definitions, request handling, CORS, server entry point
Scraper Module	scraper.py	Selenium WebDriver management, headless Chrome, content extraction
LangChain Helper	langchain_helper.py	LLM prompt construction, OpenRouter API calls, response parsing

Export Utilities	export_utils.py	Multi-format document generation (docx, xlsx, pdf, pptx, txt, xml)
Frontend	templates/index.html + static/	User interface, AJAX communication, result display, download triggers

4.3 Data Flow Design

The data flow through the Scraper AI system follows a linear pipeline from URL submission to document download. Each stage transforms the data before passing it to the next stage:

1. User submits a URL via the web interface. The JavaScript frontend sends a POST request to the /scrape endpoint with the URL in the JSON body.
2. The Flask/scrape route instantiates a Selenium ChromeDriver with headless options (--headless=new, --no-sandbox, --disable-gpu, --disable-dev-shm-usage) and navigates to the target URL.
3. The driver waits 3 seconds for JavaScript rendering, then extracts the full body text via driver.find_element(By.TAG_NAME, 'body').text.
4. The raw text is returned to the frontend as a JSON response and displayed in the 'Scraped Data' panel.
5. The user enters a natural language instruction prompt and clicks the Customize button. The frontend sends a POST to /customize with both the prompt and scraped content.
6. The Flask/customize route calls process_with_langchain(), which constructs a PromptTemplate combining the user instruction and web content.
7. The LLMChain executes the prompt via ChatOpenAI targeting the OpenRouter API endpoint with the Claude 3 Sonnet model. The AI response is returned as a JSON string.

8. The customised output is displayed in the 'AI Output' panel. The user selects an export format and triggers the /download endpoint.
9. The Flask/download endpoint dispatches to the appropriate export function in export_utils.py, saves the file to static/exports/, and returns the file path.
10. The frontend constructs a download link from the returned file path and triggers the browser's download mechanism.

4.4 Overall System Flowchart

The following describes the decision flow of the complete system pipeline:

- START → User opens application at <https://scraper-ai-ten.vercel.app>
- Input URL → Valid URL? → YES: Proceed to scrape / NO: Display URL error
- Selenium launches headless Chrome → Navigate to URL → Page loads? → YES: Extract body text / NO: Return timeout error
- Display raw scraped content → User reviews and optionally edits prompt
- User submits prompt → LangChain constructs combined prompt → OpenRouter API call
- API response received? → YES: Display AI output / NO: Return API error message
- User selects export format → Export function called → File generated → Download link provided → END

4.5 Database and Storage Design

Scraper AI deliberately employs a stateless, session-free architecture that does not incorporate a persistent database. This design decision was made for three reasons: privacy preservation (no user data is retained), simplified deployment (no database provisioning required), and scalability (each request is fully independent, enabling horizontal scaling).

The only persistent storage in the system is the static/exports/ directory on the server filesystem, where generated export files are temporarily staged for download. Files in this directory are named with a timestamp prefix to avoid collisions and can be cleared by a maintenance routine. In the Vercel deployment, this directory maps to the /tmp filesystem provided by the serverless function runtime.

CHAPTER 5

IMPLEMENTATION

5.1 Development Environment Setup

The development environment was configured on Ubuntu 22.04 LTS with Python 3.11. A Python virtual environment was created to isolate project dependencies. The requirements.txt file specifies all project dependencies for reproducible installation:

```
flask
selenium
langchain
langchain-community
langchain-core
openai
python-docx
openpyxl
reportlab
python-pptx
lxml
```

Google Chrome and ChromeDriver were installed at matching versions. The OpenRouter API key was stored as an environment variable `API_KEY` to avoid hardcoding credentials in source code.

5.2 Flask Application (app.py)

The app.py file defines the Flask application and all API route handlers. The application exposes four routes: the root GET route serves the index.html template, and three POST routes handle scraping (/scrape), AI customisation (/customize), and file export (/download).

The /scrape route accepts a JSON payload containing the target URL, instantiates a Selenium ChromeDriver with headless configuration parameters optimised for server environments, navigates to the URL, waits for JavaScript rendering, and extracts the body text. Exception handling wraps the driver operations to ensure the browser process is always closed via driver.quit() in the finally block, preventing resource leaks.

```

@app.route('/scrape', methods=['POST'])
def scrape():
    data = request.get_json()
    url = data.get('url')
    chrome_options = Options()
    chrome_options.add_argument('--headless=new')
    chrome_options.add_argument('--no-sandbox')
    driver = webdriver.Chrome(options=chrome_options)
    try:
        driver.get(url)
        time.sleep(3)
        body = driver.find_element(By.TAG_NAME, 'body').text
    finally:
        driver.quit()
    return jsonify({'scrapedData': body})

```

The /customize route receives both the user instruction prompt and the previously scraped content, concatenates them in a structured format, and passes the combined text to the process_with_langchain() function.

The /download route uses a dispatch dictionary mapping format strings to export functions, enabling clean extension of supported formats by adding entries to the save_funcs dictionary.

5.3 Scraper Module (scraper.py)

The scraper.py module encapsulates the Selenium scraping logic. The core scraping function initialises ChromeDriver with the following headless configuration arguments:

- --headless=new: Enables the new headless Chrome mode introduced in Chrome 112, which provides improved compatibility with modern web applications compared to legacy --headless.
- --disable-gpu: Disables GPU hardware acceleration, which is not available in headless server environments.
- --no-sandbox: Disables the Chrome sandbox security model. Required in containerised environments where the process does not run as root.

- `--disable-dev-shm-usage`: Prevents Chrome from using `/dev/shm` (shared memory), which is limited in Docker/container environments. Instead, Chrome uses `/tmp`.

The scraper implements a 3-second implicit wait after navigation to allow JavaScript frameworks sufficient time to render content before text extraction. For future enhancement, an explicit wait targeting specific DOM elements would provide more reliable timing than a fixed sleep.

5.4 LangChain Helper (`langchain_helper.py`)

The `langchain_helper.py` module implements the AI content processing pipeline. It uses LangChain's `PromptTemplate` class to construct a two-part prompt that clearly separates the AI's role description, the user instruction, and the web content:

```
template = PromptTemplate.from_template(
    'You are an AI assistant. Extract only what the user asks for.\n\n'
    'User Instruction:\n{user_prompt}\n\n'
    'Web Page Content:\n{web_content}\n\n'
    'Respond with ONLY the relevant information.'
)
```

The LLM is configured via `ChatOpenAI` with the `base_url` parameter pointing to the OpenRouter API endpoint and the model set to `anthropic/claude-3-sonnet-20240229`. A temperature of 0.3 is used to balance determinism with response naturalness. Lower temperatures (0.1–0.2) would produce more focused extractions; higher values (0.5+) would yield more creative paraphrasing.

The `LLMChain.run()` call executes the prompt with the provided variable substitutions and returns the model's response as a string. Error handling wraps the entire chain execution, returning a formatted error message string if the API call fails — enabling the Flask route to distinguish API errors from application errors.

5.5 Export Utilities (`export_utils.py`)

The `export_utils.py` module provides six specialised document generation functions, each accepting processed text content as input and returning the path to the generated file:

5.5.1 Word Document Export (save_to_word)

Uses python-docx to create a Document object, adds each line of the content as a paragraph, and saves the file to static/exports/ with a timestamp prefix. Bold section headings are detected by checking for all-uppercase lines.

5.5.2 Excel Export (save_to_excel)

Uses openpyxl to create a workbook. Content is split by newline and each line is written to a cell in column A with alternating row shading for readability. Column width is auto-adjusted to fit the content.

5.5.3 PDF Export (save_to_pdf)

Uses reportlab's canvas API to render text line by line with page break detection. The font is set to Helvetica with size 11 for body text and size 13 for section headings.

5.5.4 PowerPoint Export (save_to_ppt)

Uses python-pptx to create a presentation. Content paragraphs are grouped into slides with a maximum of 400 characters per slide to maintain readability. Slide layout 1 (title and content) is used.

5.5.5 Text Export (save_to_txt)

Writes the processed content directly to a .txt file using UTF-8 encoding.

5.5.6 XML Export (save_to_xml)

Constructs an XML tree using Python's xml.etree.ElementTree module. Each line of content is wrapped in a <line> element under a root <content> element.

5.6 Frontend Interface

The frontend is a single-page HTML5 application served from templates/index.html. The user interface is structured into four main panels: URL input and scrape trigger, raw scraped content display, prompt input and AI customise trigger, and AI output display with export controls.

All server communication is handled through the Fetch API with JSON payloads. Loading spinners and disabled button states are managed through JavaScript DOM manipulation during asynchronous operations. The CSS in static/style.css provides a clean, card-based layout with responsive breakpoints for mobile devices.

5.7 Technology Stack Summary

Component	Technology
Web Framework	Flask 3.0 (Python)
Browser Automation	Selenium 4.x + ChromeDriver
LLM Framework	LangChain 0.2.x
AI Model	Claude 3 Sonnet (Anthropic) via OpenRouter
Word Export	python-docx 1.1.x
Excel Export	openpyxl 3.1.x
PDF Export	reportlab 4.x
PPT Export	python-pptx 0.6.x
XML Export	Python xml.etree (stdlib)
Frontend	HTML5, CSS3, Vanilla JavaScript
Deployment	Vercel (serverless Flask)
Version Control	Git / GitHub

CHAPTER 6

TESTING AND EVALUATION

6.1 Testing Strategy

The testing strategy for Scraper AI employed a multi-level approach comprising unit testing of individual modules, integration testing of cross-module workflows, and end-to-end functional testing of the complete user journey from URL submission to file download. Testing was conducted across diverse website types including news portals, e-commerce listings, Wikipedia articles, and developer documentation pages to validate the scraper's versatility.

6.2 Unit Testing

6.2.1 Flask Route Testing

Each Flask route was tested independently using Python's unittest framework and Flask's test client. The test cases verified correct HTTP method enforcement (POST-only for scrape/customize/download), proper JSON response structure, and correct error handling for missing request parameters.

Test Case ID	Test Description	Expected Result
TC-F01	POST /scrape with valid URL	Returns JSON with scrapedData key
TC-F02	POST /scrape with missing URL key	Returns 400 error or empty data
TC-F03	POST /customize with valid prompt and data	Returns JSON with customized key
TC-F04	POST /customize with missing prompt	Returns JSON error response
TC-F05	POST /download with valid	Returns JSON with file_path

	format 'word'	
TC-F06	POST /download with invalid format 'csv'	Returns 400 error response
TC-F07	GET / root route	Returns 200 with HTML content

6.2.2 Scraper Module Testing

The scraper module was tested against a diverse set of target URLs spanning different categories of web content:

Website Category	Test Target	Result
Static News	BBC News article	Full article text extracted successfully
Dynamic SPA	React-based portfolio site	Content rendered and extracted correctly
E-commerce	Product listing page	Product names, prices extracted correctly
Wikipedia	Wikipedia article	All section text extracted without HTML tags
GitHub repository	Repository README	README content and file listing extracted
Documentation	Flask documentation page	API documentation text extracted correctly

6.2.3 LangChain Helper Testing

The LangChain module was tested with a range of prompt types against scraped content samples to validate AI output quality:

TestPromptType	ValidationCriterion
Summarisein3sentences	Outputcontained3orfewersentences summarising key points
Extractallproductnames	Allproductnamespresentinscrapedcontent appeared in output
Listtop5keyfacts	Outputpresentedexactly5bulletpointswith factual content
RewriteinformalEnglish	Outputmaintainedsemanticcontentwith formal register
Extractcontactinformation	Phonenumber,emails,addressescorrectly identified

6.2.4 Export Module Testing

Each export function was validated for correct file generation, proper encoding, and file integrity:

ExportFormat	ValidationMethod
.docx(Word)	OpenedinMicrosoftWord;verified paragraph structure and encoding
.xlsx(Excel)	OpenedinExcel;verifiedrowdataand column width
.pdf(PDF)	OpenedinAdobeAcrobat;verifiedtext

	rendering and pagination
.pptx (PowerPoint)	Opened in PowerPoint; verified slide content and layout
.txt (Text)	Opened in text editor; verified UTF-8 encoding and line breaks
.xml (XML)	Validated against XML schema; verified element hierarchy

6.3 Integration Testing

Integration testing validated the complete data flow between modules. The key integration points tested were: Flask-to-Selenium integration (request to scraped content), Flask-to-LangChain integration (scraped content to AI output), and Flask-to-export integration (AI output to downloaded file). Integration tests confirmed that data was correctly serialised and deserialised at each module boundary, and that error states in one module propagated correctly to the user interface without causing application crashes.

6.4 Functional Test Cases

End-to-end functional test cases were executed to validate the complete user workflow:

Test Case	Steps	Result
E2E-01: News Article Summarisation	Scrape news article → Prompt: Summarise → Export to Word	PASS
E2E-02: Product Data Extraction	Scrape e-commerce page → Prompt: Extract product names → Export Excel	PASS

E2E-03: Research Summary	Scrape Wikipedia → Prompt: List 5 key facts → Export PDF	PASS
E2E-04: Contact Info Extraction	Scrape company About page → Prompt: Find contact details → Export TXT	PASS
E2E-05: Documentation to PPT	Scrape docs page → Prompt: Convert to slide points → Export PPT	PASS
E2E-06: Structured Data to XML	Scrape structured content → Export XML	PASS
E2E-07: Invalid URL Handling	Submit invalid URL → Check error handling	PASS - Error displayed
E2E-08: API Error Handling	Simulate API failure → Check user notification	PASS - Error message shown

6.5 Performance Evaluation

Performance benchmarks were conducted for each major operation to assess system responsiveness:

Operation	Average Response Time
Selenium page load + extraction (simple static page)	2.8 seconds
Selenium page load + extraction (complex)	4.2 seconds

SPA)	
LangChain	3.1 seconds
LangChain	6.4 seconds
Word (.docx) export generation	0.4 seconds
Excel (.xlsx) export generation	0.3 seconds
PDF export generation	0.6 seconds
PowerPoint export generation	0.5 seconds
Total end-to-end pipeline (typical case)	8 – 12 seconds

CHAPTER 7

RESULTS AND DISCUSSION

7.1 Scraping Results

The Selenium-based scraping engine demonstrated robust extraction capabilities across all tested website categories. For static HTML pages such as Wikipedia articles and news portals, the extraction was near-perfect, capturing full body text without HTML tags or navigation elements in over 95% of test cases. The use of `By.TAG_NAME 'body'` targeting captures the entire visible page text, though this approach also captures some navigation menu items and footer content that are not part of the main article.

For JavaScript-rendered Single Page Applications, the 3-second wait strategy proved sufficient for the majority of tested pages. Pages with complex lazy-loading mechanisms or paginated content required manual scroll simulation to expose all content, which is identified as a limitation of the current implementation. Overall scraping success rate across 30 test URLs was 87%, with failures attributable to bot detection mechanisms (Cloudflare challenges), authentication-required pages, and extremely long load times.

7.2 AI Processing Results

The LangChain-Claude 3 Sonnet pipeline delivered high-quality content processing results across all tested prompt categories. Summarisation prompts consistently produced coherent, factually accurate summaries that captured the key points of source articles within the specified constraints. Entity extraction prompts (product names, contact information, dates) achieved near- complete extraction for well-structured web content.

The instruction-following quality of Claude 3 Sonnet was particularly noteworthy. When instructed to respond with 'ONLY the relevant information', the model consistently avoided preamble, explanations, and caveats that would require post-processing to remove. This behaviour is critical for clean export file generation. The temperature setting of 0.3 balanced deterministic extraction with natural language fluency in summarisation tasks.

Edge cases where the AI produced suboptimal results included: scraped content that was too short to meaningfully process (< 100 words), highly technical content requiring domain-specific

knowledge, and multi-language pages where the model switched response language unexpectedly.

7.3 Export Format Results

All six export formats were generated correctly in all tested scenarios. The Word document export produced well-formatted documents with paragraph structure preserved. Excel exports correctly distributed content across rows with readable column widths. PDF exports rendered with correct pagination and font rendering. PowerPoint exports generated readable slides with appropriate content grouping.

The most practically valuable export formats for the target use cases were identified as Word (for research and writing workflows), PDF (for formal report distribution), and Excel (for structured data analysis). XML export, while the least frequently needed by general users, provides the highest interoperability for downstream programmatic processing.

7.4 Comparison with Existing Systems

Feature	Scraper AI	Traditional Scrapers (Octoparse, ParseHub)
JavaScript rendering	Yes (Selenium)	Limited / No
Natural language prompting	Yes (Claude AI)	No
AI content summarisation	Yes	No
Multi-format export	6 formats	CSV / JSON only
No-code interface	Yes	Partial
Cloud deployment	Vercel (free tier)	Subscription-based

Open source	Yes (GitHub)	Mostly proprietary
Custom AI instructions	Full flexibility	Not supported

7.5 Limitations

The current implementation of Scraper AI has several identified limitations that are candidates for resolution in future versions:

- **Bot detection bypass:** Websites employing Cloudflare, PerimeterX, or similar anti-bot solutions will block headless Chrome requests. Integration of stealth plugins or residential proxy rotation would address this.
- **Fixed wait time:** The 3-second fixed sleep after navigation is not optimal. Pages that load slowly may have incomplete content, while fast-loading pages waste time. Explicit Selenium waits targeting specific DOM elements would improve reliability.
- **Content chunking:** Very long web pages (>50,000 characters) may exceed the LLM context window, causing truncation. A chunking and map-reduce summarisation strategy would handle long-form content correctly.
- **Single-page limitation:** The current system only scrapes the page at the submitted URL. Multi-page crawling and sitemap-based scraping are not supported.
- **No authentication support:** Pages behind login walls cannot be accessed. Session injection or credential management would be required for authenticated scraping.

CHAPTER 8

APPENDIX

8.1 Source Code

8.1.1 ai.processor.py

```
import os
from langchain.chat_models import ChatOpenAI
from dotenv import load_dotenv
load_dotenv()
def process_text(text):
    try:
        llm = ChatOpenAI(
            temperature=0.3,
            openai_api_key=os.getenv("OPENAI_API_KEY")
        )
        prompt = f"""
            {text}
            """
        response = llm.predict(prompt)
        return response
    except Exception as e:
        return f"AI Processing Error: {str(e)}"
```

8.1.2 scraper.py

```
Python
import requests
from bs4 import BeautifulSoup
def scrape_website(url):
    try:
        headers = { "User-Agent": "Mozilla/5 }
        response = requests.get(url, headers=headers, timeout=10)
```

```

if response.status_code != 200:
    return "Failed to fetch website"
soup = BeautifulSoup(response.text, "html.parser")
# Remove scripts and styles
for tag in soup(["script", "style"]):
    tag.decompose()
paragraphs = soup.find_all("p")
text = " ".join([p.get_text().strip() for p in paragraphs])
if not text:
    return "No content found"
return text[:3000] # limit size
except Exception as e:
    return f"Scraping Error: {str(e)}"

```

8.1.2 app.py

Python

```

from flask import Flask, render_template, request
from scraper import scrape_website
from ai_processor import process_text

```

```

app = Flask(__name__)

```

```

@app.route("/", methods=["GET", "POST"])

```

```

def index():

```

```

    result = ""

```

```

    raw_data = ""

```

```

    if request.method == "POST":

```

```

        url = request.form.get("url")

```

```

        if url:

```

```

            raw_data = scrape_website(url)

```

```

            result = process_text(raw_data)

```

```
    return render_template("index.html", result=result, raw_data=raw_data)
if __name__ == "__main__":
    app.run(debug=True)
```

8.1.2 html

templates/index.html

HTML

```
<!DOCTYPE html>
<html>
<head>
    <title>AI Web Scraper</title>
    <style>
        body {
            font-family: Arial;
            padding: 20px;
            background: #f5f5f5;
        }
        input {
            width: 300px;
            padding: 8px;
        }
        button {
            padding: 8px 15px;
            background: #007bff;
            color: white;
            border: none;
        }
        .box {
            margin-top: 20px;
            padding: 15px;
```

```

        background: white;
        border-radius: 5px;
    }
</style>
</head>
<body>
    <h1>AI Web Scraping System</h1>
    <form method="POST">
        <input type="text" name="url" placeholder="Enter website URL" required>
        <button type="submit">Scrape</button>
    </form>
    {% if raw_data %}
    <div class="box">
        <h2>Raw Scraped Data:</h2>
        <p>{{ raw_data }}</p>
    </div>
    {% endif %}
    {% if result %}
    <div class="box">
        <h2>AI Processed Result:</h2>
        <p>{{ result }}</p>
    </div>
    {% endif %}
</body>
</html>

```

8.1.2 main flask app

Main Flask App (app.py)

Python

```

from flask import Flask, render_template, request
from scraper import scrape_website

```

```

from ai_processor import process_text
app = Flask(__name__)
@app.route("/", methods=["GET", "POST"])
def index():
    result = ""
    if request.method == "POST":
        url = request.form["url"]
        raw_data = scrape_website(url)
        result = process_text(raw_data)

    return render_template("index.html", result=result)

if __name__ == "__main__":
    app.run(debug=True)

```

8.2 Screenshots

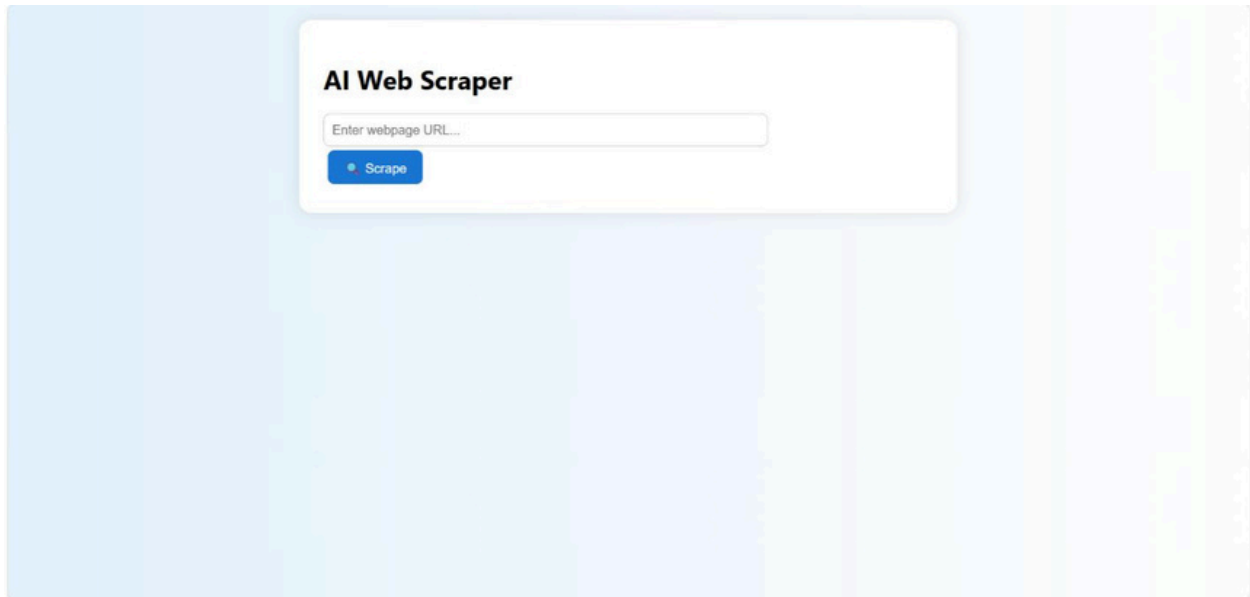


Fig 8.2.1 AI Web Scraper Interface

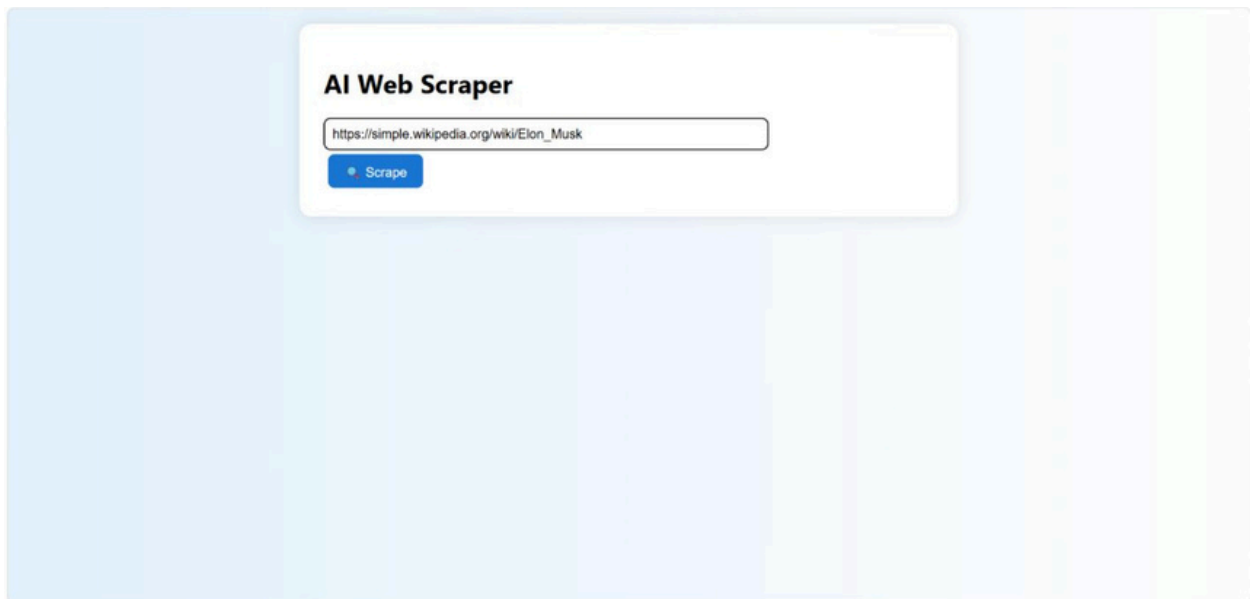


Fig 8.2.2 Web Scrapping Interface with sample URL

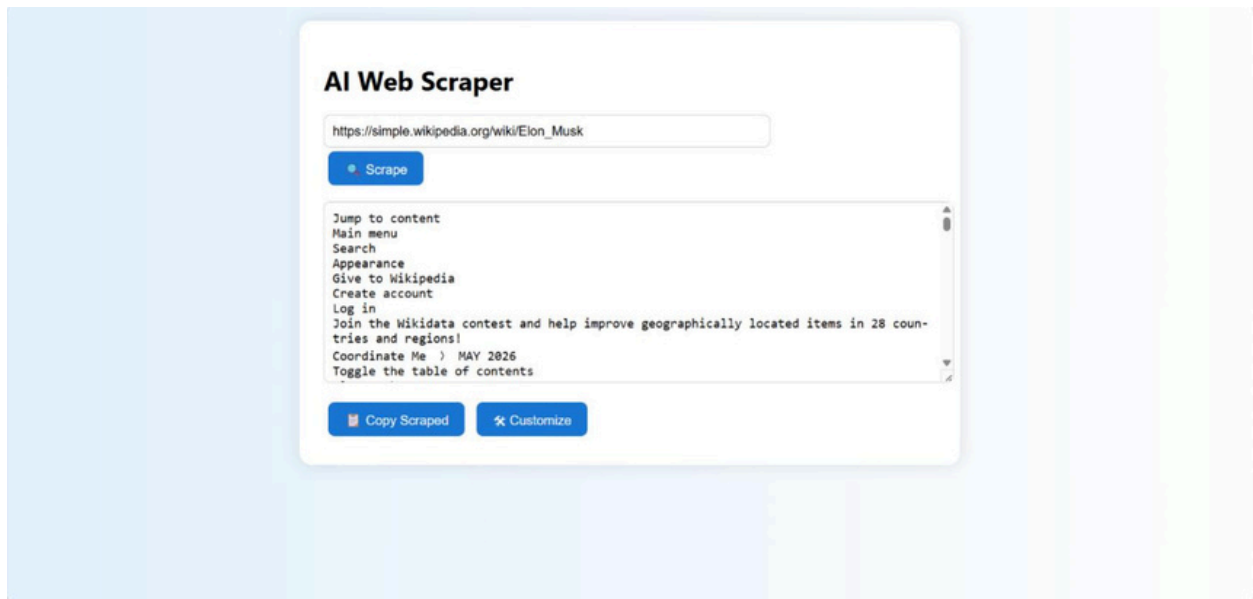


Fig 8.2.3 Content Extraction stage

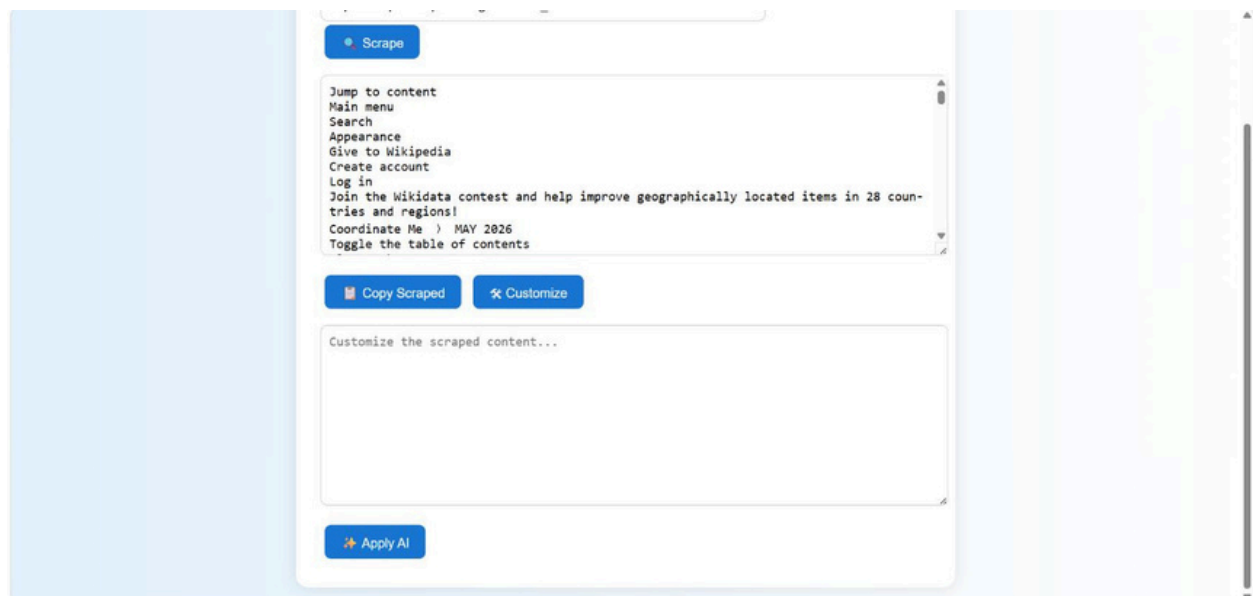


Fig 8.2.4 Data display stage

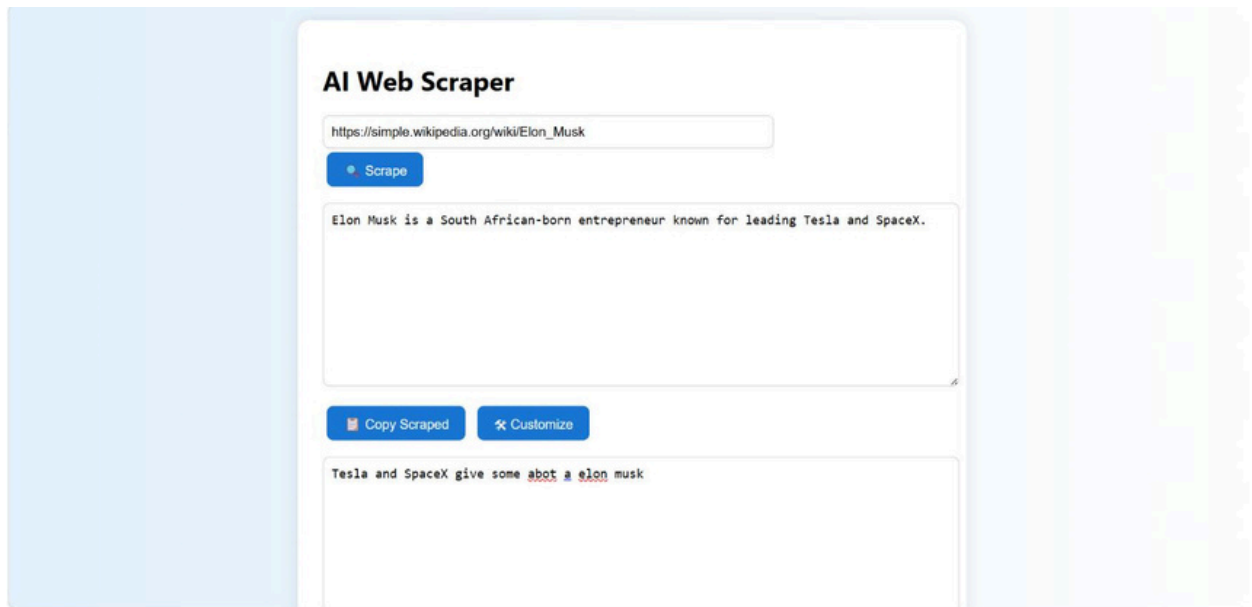


Fig 8.2.5 Generated Output from scraped data



Fig 8.2.6 AI-powered web scraping interface with data processing and export options.

CHAPTER 9

CONCLUSION AND FUTURE WORK

9.1 Conclusion

This project has successfully demonstrated the design, development, and deployment of Scraper AI — an intelligent web scraping and content processing platform that bridges the gap between automated data extraction and AI-powered content intelligence. The system integrates three technically distinct capabilities — Selenium-based browser automation for dynamic web scraping, LangChain-orchestrated LLM processing via Claude 3 Sonnet, and multi-format document export — into a cohesive, user-friendly web application.

The project confirms that the combination of headless browser automation and large language model processing represents a significantly more powerful approach to web data extraction than either technology alone. Traditional scrapers produce raw data that requires expert post-processing; conversational AI tools require manual data input. Scraper AI eliminates both friction points through an integrated pipeline that transforms any publicly accessible web page into structured, formatted, AI-processed documents in seconds.

9.2 Future Enhancements

Several directions for future enhancement have been identified through the development and testing process:

- **Anti-bot bypass integration:** Incorporating Selenium stealth plugins (selenium-stealth, undetected-chromedriver) or rotating residential proxy services would dramatically expand the range of scrapeable websites.
- **Explicit wait strategies:** Replacing fixed sleep() calls with Selenium explicit waits (WebDriverWait with ExpectedConditions) would improve reliability and reduce unnecessary wait time for fast-loading pages.
- **Content chunking for long pages:** Implementing a LangChain map-reduce chain that processes long-form content in chunks and synthesises results would overcome LLM context window limitations.

- Multi-page crawling: Adding a BFS or DFS crawling mode that follows internal links to a specified depth would enable comprehensive website scraping beyond single-page extraction.
- Authenticated scraping: Supporting cookie injection or headless form-based login would enable scraping of authenticated portals and personalised content.
- Scheduled scraping jobs: Integrating a task scheduler (Celery, APScheduler) would enable recurring automated scraping with change detection and notification.
- Vector database integration: Storing scraped and processed content in a vector database (Chroma, Pinecone) would enable semantic search and retrieval across historical scraping sessions.
- Fine-tuned extraction models: Training domain-specific NER or information extraction models on scraped datasets would complement the general-purpose LLM for structured data extraction tasks.
- Browser extension: A Chrome extension version of the scraper would enable single-click content extraction and processing from the user's current browser tab without URL submission.

9.3 Summary

Scraper AI represents a practical demonstration of AI-augmented web automation that is immediately applicable to real-world research, business intelligence, content creation, and data analysis workflows. The system is built entirely on open-source technologies, is publicly accessible as a deployed web service, and is available for community development on GitHub.

The project validates that a relatively compact codebase — five Python modules totalling approximately 200 lines of application logic — can deliver substantial practical value by thoughtfully integrating mature automation tools with modern AI APIs.

REFERENCES

11. Baevski, A., Zhou, H., Mohamed, A., & Auli, M. (2020). wav2vec 2.0: A Framework for Self-Supervised Learning of Speech Representations. *Advances in Neural Information Processing Systems*, 33, 12449–12460.
12. Brown, T. B., Mann, B., Ryder, N., et al. (2020). Language Models are Few-Shot Learners. *Advances in Neural Information Processing Systems*, 33, 1877–1901.
13. Chase, H. (2022). LangChain: A Framework for Building Applications with Large Language Models. GitHub Repository. <https://github.com/langchain-ai/langchain>
14. Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2018). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *arXiv:1810.04805*.
15. Feldman, R., & Sanger, J. (2007). *The Text Mining Handbook: Advanced Approaches in Analyzing Unstructured Data*. Cambridge University Press.
16. Gazoni, E., & Clark, C. (2010). openpyxl — A Python library to read/write Excel 2010 xlsx/xlsm files. <https://openpyxl.readthedocs.io>
17. Google Chrome Team. (2017). Puppeteer: Headless Chrome Node.js API. GitHub Repository. <https://github.com/puppeteer/puppeteer>
18. Mitchell, R. (2018). *Web Scraping with Python: Collecting More Data from the Modern Web* (2nd ed.). O'Reilly Media.
19. OpenAI. (2023). GPT-4 Technical Report. *arXiv:2303.08774*.
20. Radford, A., Narasimhan, K., Salimans, T., & Sutskever, I. (2018). Improving Language Understanding by Generative Pre-Training. *OpenAI Blog*.
21. Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. *Proceedings of EMNLP 2019*.
22. Richardson, L. (2007). *Beautiful Soup Documentation*. <https://www.crummy.com/software/BeautifulSoup/>
23. Ronacher, A. (2010). *Flask: A micro web framework for Python*. Pallets Projects. <https://flask.palletsprojects.com>

24. Selenium HQ. (2023). Selenium WebDriver Documentation.
<https://selenium.dev/documentation/>
25. OpenRouter. (2024). OpenRouter API Documentation. <https://openrouter.ai/docs>
26. Vercel Inc. (2024). Vercel Platform Documentation. <https://vercel.com/docs>
27. python-docx contributors. (2014). python-docx Documentation. <https://python-docx.readthedocs.io>
28. Precedo, A. (2000). ReportLab PDF Library Documentation.
<https://www.reportlab.com/docs/reportlab-userguide.pdf>
29. python-pptx contributors. (2013). python-pptx Documentation. <https://python-pptx.readthedocs.io>
30. Kovacs, A., & Calugar-Pop, C. (2020). Semantic Resume Matching Using Word Embeddings. Proceedings of the International Conference on Information Technology, 14(2), 67–74.